

Putting the DD Statement to Work





PUTTING THE DD STATEMENT TO WORK



You've written JCL for a job that does nothing. Let's write a job that does something.

As a batch processing system, the mainframe spends a lot of time processing data sets or files in batch mode.

There are several different types of data sets and each one is managed by a bespoke Access Method.

The JCL statement that controls how the system manipulates data sets is the Data Definition statement, or DD statement.

MORE SECRETS OF THE ISPF EDITOR

In this module, we'll explore JCL in more depth. We'll be coding JCL for several jobs. To make that easier, let's take a look at some of the capabilities of the ISPF Editor.

The Editor offers a couple of ways to create Library members based on existing members.

One way is to open a member, select a block of lines, and copy or move those lines into a new member.

Another way is to open a new member, which may be new and empty, and then copy the contents of an existing member into it.

We're going to use these techniques to create a JOB statement member that we can use when creating new jobs, and to initialize those new jobs by copying the JOB statement into new members.



CREATING A JCL SNIPPET FOR OUR JOB CARD

Enter CREATE command followed by new member name

Enter block copy command in the line command area. That's CC on the first line to copy and another CC on the last line to copy.

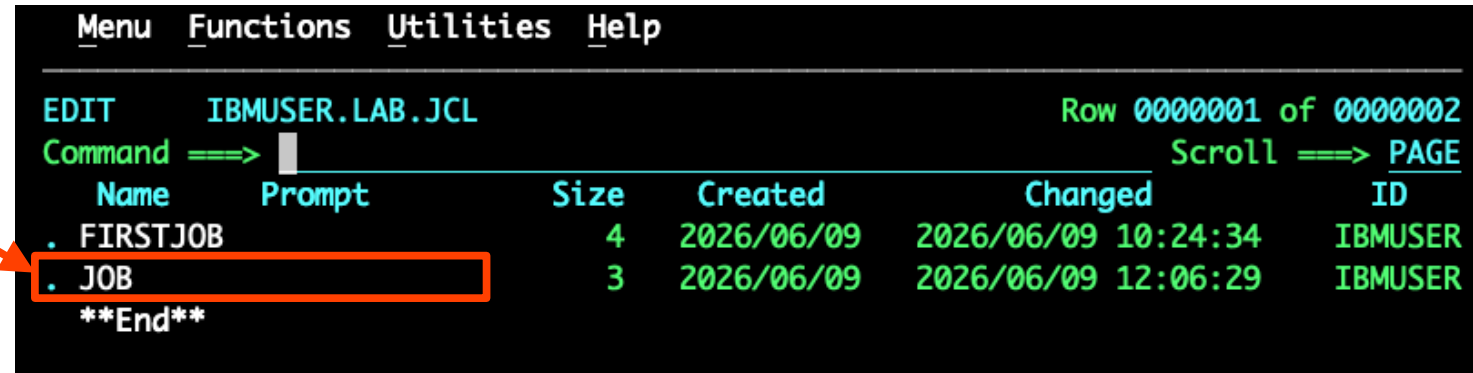
```
File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(FIRSTJOB) - 01.01      Columns 00001 00072
Command ==> create job                          Scroll ==> CSR
***** ***** Top of Data *****
cc 001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
cc 003 //          NOTIFY=&SYSUID
000004 //NADA      EXEC PGM=IEFBR14
***** ***** Bottom of Data *****
```

```
File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(FIRSTJOB) - 01.01      Member JOB created
Command ==>                                     Scroll ==> CSR
***** ***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //NADA      EXEC PGM=IEFBR14
***** ***** Bottom of Data *****
```

Press the Enter key, and it tells you that it created a new member named JOB in the same Library.

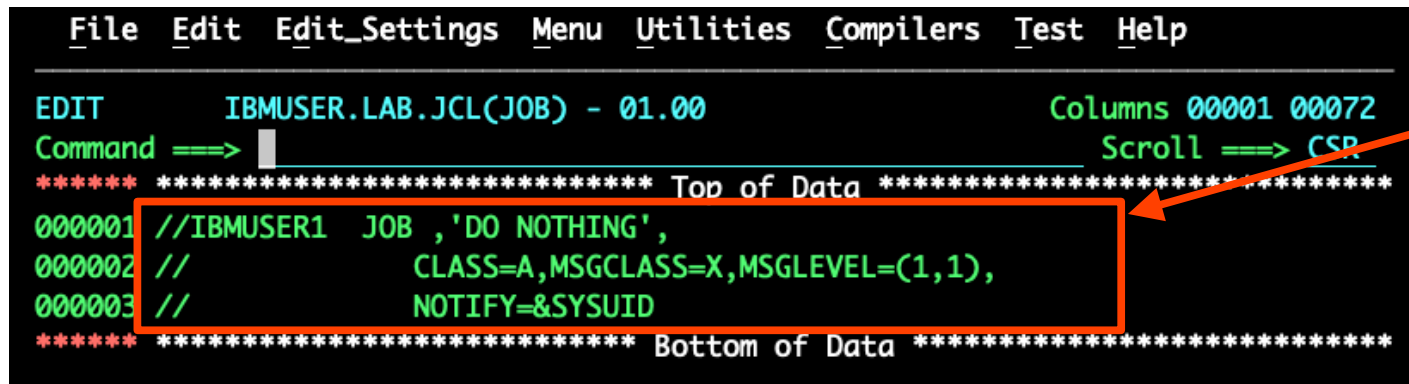
CREATING A JCL SNIPPET FOR OUR JOB CARD

When we exit with F3 or F12 we see there's a member in our Library named JOB.



The screenshot shows the ISPF library panel for the library IBMUSER.LAB.JCL. The member JOB is selected and highlighted with an orange box. An orange arrow points from the text on the left to this member. The panel displays a table of members with columns for Name, Prompt, Size, Created, Changed, and ID. The member JOB has a size of 3 and was created on 2026/06/09 at 12:06:29.

Name	Prompt	Size	Created	Changed	ID
. FIRSTJOB		4	2026/06/09	2026/06/09 10:24:34	IBMUSER
. JOB		3	2026/06/09	2026/06/09 12:06:29	IBMUSER



The screenshot shows the ISPF editor for the member JOB. The editor displays the JCL snippet for the member, which is highlighted with an orange box. An orange arrow points from the text on the right to this snippet. The JCL snippet consists of three lines: //IBMUER1 JOB , 'DO NOTHING', // CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1), and // NOTIFY=&SYSUID. The editor also shows the top and bottom of the data area.

```
***** Top of Data *****
000001 //IBMUER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
***** Bottom of Data *****
```

When we select member JOB we see it contains the lines we selected from member FIRSTJOB – our JOB statement and nothing else.

CREATING A JCL MEMBER FOR A NEW JOB

To create a member for a new job, we back out to the Edit Entry Panel and enter the name of the new member.

```

Menu  RefList  RefMode  Utilities  Help

Edit Entry Panel

Command ==>

ISPF Library:
Project . . . IBMUSER
Group . . . LAB
Type . . . JCL
Member . . . crqsam (Blank or pattern for member selection list)

Other Partitioned, Sequential or VSAM Data Set, or z/OS UNIX file:
Name . . .
Volume Serial . . (If not cataloged)
  
```

```

File Edit Edit_Settings Menu Utilities Compilers Test Help

EDIT IBMUSER.LAB.JCL(CRQSAM) - 01.00 Columns 00001 00072
Command ==> copy job Scroll ==> CSR
a***** Top of Data *****
  
```

The Editor panel opens with several blank lines, as we saw before. We type the command COPY followed by the name of the member we want to copy, and type an A (for "after") in the line command area next to the line of asterisks, where it says "Top of Data".

CREATING A JCL MEMBER FOR A NEW JOB

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.00      Member JOB copied
Command ==>                               Scroll ==> CSR
***** ***** Top of Data *****
000001 //IBMUSER1  JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
***** ***** Bottom of Data *****

```

The COPY command populates the new member with the contents of member JOB. Now we have the beginnings of a new job.

The benefit of doing this is we never have to code the JOB statement from scratch, with the attendant risk of mis-keying something.

The name of the member, CRQSAM, suggests we want the job to create a QSAM data set. Oldtimers often refer to data sets by their Access Method rather than the technically-correct data set attribute, such as Physical Sequential (PS). Physical Sequential data sets will be processed by QSAM (Queued Sequential Access Method) unless we specifically tell the system to use BSAM. So, we'll call it a "QSAM file."

But there isn't enough JCL in here to create anything. We don't even have a minimal job that will run, because there are no steps.

We need to add an EXEC statement to execute the do-nothing utility, IEFBR14. We'll let JES do the heavy lifting to allocate the new data set.

Then we'll need a DD statement to define the attributes of the data set we want to allocate.

We also might want to change the text on the JOB statement so it doesn't say 'DO NOTHING'. We're going to do 'something' this time.

ATTRIBUTES OF A SEQUENTIAL DATA SET

You remember how to allocate a data set via ISPF option 3.2. Let's use that panel as a guide to code our DD statement parameters.

Allocate New Data Set	
Command ==>	
Data Set Name	IBMUUSER.QSAM1
Management class . . .	(Blank for default management class)
Storage class	SCBASE (Blank for default storage class)
Volume serial	USRV51 (Blank for system default volume) **
Multiple Volumes	(Enter '/' to select option)
Device type	(Generic unit or device address) **
Data class	(Blank for default data class)
Space units	TRACK (BLKS, TRKS, CYLS, KB, MB, BYTES or RECORDS)
Average record unit	(M, K, or U)
Primary quantity . .	5 (In above units)
Secondary quantity	2 (In above units)
Directory blocks . .	0 (Zero for sequential data set) *
Record format	FB
Record length	80
Block size	16000
Data set name type	(LIBRARY, PDS, LARGE, BASIC, * EXTREQ, EXTPREF or blank)

ATTRIBUTES OF A SEQUENTIAL DATA SET

The screenshot displays a JCL editor window titled 'IBMUSER.LAB.JCL(CRQSAM) - 01.01'. The JCL code is as follows:

```

000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1        DD DSN=&SYSUID..QSAM1,

```

The right panel shows the 'Allocate New Data Set' dialog. The 'Data Set Name' field is highlighted with an orange box and contains 'IBMUSER.QSAM1'. Other fields include 'Management class', 'Storage class' (SCBASE), 'Volume serial' (USRVS1), 'Device type', 'Data class', 'Space units' (TRACK), 'Average record unit', 'Primary quantity' (5), 'Secondary quantity' (2), 'Directory blocks' (0), 'Record format' (FB), 'Record length' (80), 'Block size' (16000), and 'Data set name type'.

The first attribute is the data set name. This is the name that will be the "key" for the system to find the data set information in the system catalog.

This is defined on the DSN= parameter of the DD statement.

Segments of data set names are separated by periods. Because &SYSUID is a symbol, we need to code two consecutive periods so JES knows this is the point of separation between the symbol and any literal text that follows it. Given the userid in this example is IBMUSER, the notation "&SYSUID..QSAM1" will resolve to "IBMUSER.QSAM1".

ATTRIBUTES OF A SEQUENTIAL DATA SET

```

File Edit Edit_Settings Menu Utilities Compilers Test H
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01      Colum
Command ==>                                     Sc
***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1         DD DSN=&SYSUID..QSAM1,
          //          DISP=(NEW,
***** Bottom of Data *****

```

When using ISPF 3.2 to allocate a data set, we don't specify a "disposition" because ISPF already knows our intention is to allocate a new data set. In JCL, we have to include a DISP= parameter on the DD statement.

The first subparameter is the *status* of the data set as of the time JES opens it. This data set doesn't exist, so its status is "new."

```

Allocate New Data Set
Command ==>
Data Set Name . . . : IBMUSER.QSAM1

Management class . . . (Blank for default management class)
Storage class . . . SCBASE (Blank for default storage class)
Volume serial . . . USRVS1 (Blank for system default volume) **
Multiple Volumes - (Enter '/' to select option)
Device type . . . (Generic unit or device address) **
Data class . . . (Blank for default data class)
Space units . . . TRACK (BLKS, TRKS, CYLS, KB, MB, BYTES
                        or RECORDS)
Average record unit - (M, K, or U)
Primary quantity . . 5 (In above units)
Secondary quantity . 2 (In above units)
Directory blocks . . 0 (Zero for sequential data set) *
Record format . . . FB
Record length . . . 80
Block size . . . 16000
Data set name type - (LIBRARY, PDS, LARGE, BASIC, *
                    EXTREQ, EXTPREF or blank)

```


ATTRIBUTES OF A SEQUENTIAL DATA SET

```

File Edit Edit_Settings Menu Utilities Compilers Test
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01      Color
Command ==>
***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1        DD DSN=&SYSUID. QSAM1,
          //          DISP=(NEW,CATLG,
***** Bottom of Data *****

```

If the job step is successful, we want JES to catalog the new data set.

```

Allocate New Data Set
Command ==>
Data Set Name . . . : IBMUSER.QSAM1

Management class . . . (Blank for default management class)
Storage class . . . SCBASE (Blank for default storage class)
Volume serial . . . USRVS1 (Blank for system default volume) **
Multiple Volumes - (Enter '/' to select option)
Device type . . . (Generic unit or device address) **
Data class . . . (Blank for default data class)
Space units . . . TRACK (BLKS, TRKS, CYLS, KB, MB, BYTES
                        or RECORDS)
Average record unit - (M, K, or U)
Primary quantity . . 5 (In above units)
Secondary quantity . 2 (In above units)
Directory blocks . . 0 (Zero for sequential data set) *
Record format . . . FB
Record length . . . 80
Block size . . . 16000
Data set name type - (LIBRARY, PDS, LARGE, BASIC, *
                    EXTREQ, EXTPREF or blank)

```

ATTRIBUTES OF A SEQUENTIAL DATA SET

```

File Edit Edit_Settings Menu Utilities Compilers Test
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01      Color
Command ==>
***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1         DD DSN=&SYSUID..QSAM1,
***** //          DISP=(NEW,CATLG,DELETE),
***** Bottom of Data *****

```

If the job step fails, we want JES to delete the new data set.

```

Allocate New Data Set
Command ==>
Data Set Name . . . : IBMUSER.QSAM1

Management class . . . (Blank for default management class)
Storage class . . . SCBASE (Blank for default storage class)
Volume serial . . . USRVS1 (Blank for system default volume) **
Multiple Volumes - (Enter '/' to select option)
Device type . . . (Generic unit or device address) **
Data class . . . (Blank for default data class)
Space units . . . TRACK (BLKS, TRKS, CYLS, KB, MB, BYTES
or RECORDS)
Average record unit - (M, K, or U)
Primary quantity . . 5 (In above units)
Secondary quantity . 2 (In above units)
Directory blocks . . 0 (Zero for sequential data set) *
Record format . . . FB
Record length . . . 80
Block size . . . 16000
Data set name type - (LIBRARY, PDS, LARGE, BASIC, *
EXTREQ, EXTPREF or blank)

```

ATTRIBUTES OF A SEQUENTIAL DATA SET

These are installation default values.

```
Allocate New Data Set

Command ==> _____

Data Set Name . . . : IBMUSER.QSAM1

Management class . . . (Blank for default management class)
Storage class . . . . SCBASE (Blank for default storage class)
Volume serial . . . . USRVS1 (Blank for system default volume) **
Multiple Volumes      - (Enter '/' to select option)
Device type . . . . . (Generic unit or device address) **

Data class . . . . . (Blank for default data class)
Space units . . . . . TRACK (BLKS, TRKS, CYLS, KB, MB, BYTES
                             or RECORDS)

Average record unit   - (M, K, or U)
Primary quantity . . 5 (In above units)
Secondary quantity    2 (In above units)
Directory blocks . . 0 (Zero for sequential data set) *
Record format . . . . FB
Record length . . . . 80
Block size . . . . . 16000
Data set name type    _____ (LIBRARY, PDS, LARGE, BASIC, *
                                EXTREQ, EXTPREF or blank)
```


ATTRIBUTES OF A SEQUENTIAL DATA SET

File Edit Edit_Settings Menu Utilities Compilers Test

EDIT IBMUSER.LAB.JCL(CRQSAM) - 01.01

Command ==>

```

***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1        DD DSN=&SYSUID..QSAM1,
000006 //          DTSP=(NEW,CATLG,DELETE),
          //          UNIT=SYSDA,
***** Bottom of Data *****

```

Allocate New Data Set

Command ==>

```

Data Set Name . . . : IBMUSER.QSAM1

Management class . . . (Blank for default management class)
Storage class . . . SCBASE (Blank for default storage class)
Volume serial . . . USRVS1 (Blank for system default volume) **
Multiple Volumes (Enter '/' to select option)
Device type . . . (Generic unit or device address) **
Data class . . . (Blank for default data class)
Space units . . . TRACK (BLKS, TRKS, CYLS, KB, MB, BYTES or RECORDS)
Average record unit (M, K, or U)
Primary quantity . . 5 (In above units)
Secondary quantity . 2 (In above units)
Directory blocks . . 0 (Zero for sequential data set) *
Record format . . . FB
Record length . . . 80
Block size . . . 16000
Data set name type (LIBRARY, PDS, LARGE, BASIC, * EXTREQ, EXTPREF or blank)

```

Regardless of the physical storage media in use, the Brain-in-a-Jar assumes all data sets are stored on DASD, Direct Access Storage Devices, as described earlier. SYSDA names a class of DASD in a system-managed pool of volumes.

The other way to specify this is to code VOL=SER=xxxxxx, naming a specific DASD volume.

ATTRIBUTES OF A SEQUENTIAL DATA SET

File Edit Edit_Settings Menu Utilities Compilers Test

EDIT IBMUSER.LAB.JCL(CRQSAM) - 01.01

Command ==>

```

***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1        DD DSN=&SYSUID..QSAM1,
000006 //          DTSP=(NEW,CATLG,DELETE),
          //          UNIT=SYSDA,
***** Bottom of Data *****

```

Allocate New Data Set

Command ==>

```

Data Set Name . . . : IBMUSER.QSAM1

Management class . . . (Blank for default management class)
Storage class . . . SCBASE (Blank for default storage class)
Volume serial . . . USRVS1 (Blank for system default volume) **
Multiple Volumes (Enter '/' to select option)
Device type . . . (Generic unit or device address) **
Data class . . . (Blank for default data class)
Space units . . . TRACK (BLKS, TRKS, CYLS, KB, MB, BYTES or RECORDS)
Average record unit (M, K, or U)
Primary quantity . . 5 (In above units)
Secondary quantity . 2 (In above units)
Directory blocks . . 0 (Zero for sequential data set) *
Record format . . . FB
Record length . . . 80
Block size . . . 16000
Data set name type (LIBRARY, PDS, LARGE, BASIC, * EXTREQ, EXTPREF or blank)

```

Regardless of the physical storage media in use, the Brain-in-a-Jar assumes all data sets are stored on DASD, Direct Access Storage Devices, as described earlier. SYSDA names a class of DASD in a system-managed pool of volumes.

The other way to specify this is to code VOL=SER=xxxxxx, naming a specific DASD volume.

ATTRIBUTES OF A SEQUENTIAL DATA SET

```

File Edit Edit_Settings Menu Utilities Compilers Test
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01      Column
Command ==>
***** ***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1        DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(NEW,CATLG,DELETE),
000007 //          UNIT=SYSDA VOL=SER=USRVS1,
***** ***** Bottom of Data *****

```

You can also code both UNIT= and VOL=SER=. The system will try to allocate the data set on the specified volume, and if there's insufficient space it will draw from the DASD pool named in the UNIT= parameter.

```

Allocate New Data Set
Command ==>
Data Set Name . . . : IBMUSER.QSAM1

Management class . . . (Blank for default management class)
Storage class . . . SCBASE (Blank for default storage class)
Volume serial . . . USRVS1 (Blank for system default volume) **
Multiple Volumes - (Enter '/' to select option)
Device type . . . (Generic unit or device address) **
Data class . . . (Blank for default data class)
Space units . . . TRACK (BLKS, TRKS, CYLS, KB, MB, BYTES
                        or RECORDS)
Average record unit - (M, K, or U)
Primary quantity . . 5 (In above units)
Secondary quantity . 2 (In above units)
Directory blocks . . 0 (Zero for sequential data set) *
Record format . . . FB
Record length . . . 80
Block size . . . 16000
Data set name type - (LIBRARY, PDS, LARGE, BASIC, *
                    EXTREQ, EXTPREF or blank)

```

ATTRIBUTES OF A SEQUENTIAL DATA SET

The screenshot shows a JCL editor window with the following JCL code:

```

EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01      Column
Command ==>
***** ***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1         DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(NEW,CATLG,DELETE),
000007 //          UNIT=SYSDA,VOL=SER=USRVS1,
000008 //          SPACE=(TRK,(5,2),RLSE),
***** ***** Bottom of Data *****
  
```

The 'Allocate New Data Set' dialog is open, showing the following options:

- Command ==>
- Data Set Name . . . : IBMUSER.QSAM1
- Management class . . . (Blank for default management class)
- Storage class . . . SCBASE (Blank for default storage class)
- Volume serial . . . USRVS1 (Blank for system default volume) **
- Multiple Volumes (Enter '/' to select option)
- Device type . . . (Generic unit or device address) **
- Data class . . . (Blank for default data class)
- Space units . . . TRACK (BLKS, TRKS, CYLS, KB, MB, BYTES or RECORDS)
- Average record unit (M, K, or U)
- Primary quantity . . 5 (In above units)
- Secondary quantity . . 2 (In above units)
- Directory blocks . . 0 (Zero for sequential data set) *
- Record format . . . FB
- Record length . . . 80
- Block size . . . 16000
- Data set name type (LIBRARY, PDS, LARGE, BASIC, * EXTREQ, EXTPREF or blank)

Unlike most other operating systems, z/OS requires us to tell it how much space is required for a new data set.

We can give a value in terms of records, blocks, bytes, kilobytes or megabytes; traditionally we would give the value in terms of tracks or cylinders, because historical DASD devices had physical tracks and cylinders. Under the covers, the system converts everything into a form the Brain-in-a-Jar can understand.

Our JCL says to allocate 5 tracks for the primary extent and 2 tracks for secondary extents. A system-defined limit on secondary extents will prevent the data set from growing indefinitely. RLSE means "release" any unused DASD space to make it available for other data sets.

ATTRIBUTES OF A SEQUENTIAL DATA SET

```

File Edit Edit_Settings Menu Utilities Compilers Test
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01      Column
Command ==>
***** ***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1         DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(NEW,CATLG,DELETE),
000007 //          UNIT=SYSDA,VOL=SER=USRVS1,
000008 //          SPACE=(TRK,(5,2),RLSE),
***** ***** Bottom of Data *****

```

There's an additional subparameter on the SPACE= parameter that specifies the number of directory blocks to include in a new PDS. We're not defining a PDS here, so we omit it.

```

Allocate New Data Set
Command ==>
Data Set Name . . . : IBMUSER.QSAM1
Management class . . . (Blank for default management class)
Storage class . . . SCBASE (Blank for default storage class)
Volume serial . . . USRVS1 (Blank for system default volume) **
Multiple Volumes (Enter '/' to select option)
Device type . . . (Generic unit or device address) **
Data class . . . (Blank for default data class)
Space units . . . TRACK (BLKS, TRKS, CYLS, KB, MB, BYTES
or RECORDS)
Average record unit (M, K, or U)
Primary quantity . . 5 (In above units)
Secondary quantity . 2 (In above units)
Directory blocks . . 0 (Zero for sequential data set) *
Record format . . . FB
Record length . . . 80
Block size . . . 16000
Data set name type (LIBRARY, PDS, LARGE, BASIC, *
EXTREQ, EXTPREF or blank)

```

ATTRIBUTES OF A SEQUENTIAL DATA SET

The image shows two ISPF panels side-by-side. The left panel is an editor for a JCL file named 'IBMUSER.LAB.JCL(CRQSAM) - 01.01'. It contains a DD statement for a sequential data set: `DD DSN=&SYSUID..QSAM1, DISP=(NEW,CATLG,DELETE), UNIT=SYSDA,VOL=SER=USRVS1, SPACE=(TRK,(5,2),RLSE), DCB=(DSORG=PS,`. The right panel is titled 'Allocate New Data Set' and shows the same data set being allocated with various attributes. Red boxes highlight specific fields in both panels: 'DCB=(DSORG=PS,' in the JCL and 'Directory blocks', 'Record length', 'Block size', and 'Data set name type' in the allocation panel. Red arrows point from the JCL box to the allocation panel boxes.

Field	Value	Description
Command	Allocate New Data Set	
Data Set Name	IBMUSER.QSAM1	
Management class		(Blank for default management class)
Storage class	SCBASE	(Blank for default storage class)
Volume serial	USRVS1	(Blank for system default volume) **
Multiple Volumes		(Enter '/' to select option)
Device type		(Generic unit or device address) **
Data class		(Blank for default data class)
Space units	TRACK	(BLKS, TRKS, CYLS, KB, MB, BYTES or RECORDS)
Average record unit		(M, K, or U)
Primary quantity	5	(In above units)
Secondary quantity	2	(In above units)
Directory blocks	0	(Zero for sequential data set) *
Record format	FB	
Record length	80	
Block size	16000	
Data set name type		(LIBRARY, PDS, LARGE, BASIC, * EXTREQ, EXTPREF or blank)

On the ISPF panel, data set organization of physical sequential is implied because no value is entered in the "Data set name type" field and the "Directory blocks" value is zero.

On the DD statement in our JCL, we have to state explicitly that the data set organization is physical sequential.

ATTRIBUTES OF A SEQUENTIAL DATA SET

```

File Edit Edit_Settings Menu Utilities Compilers Test
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01      Colum
Command ==>
***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1        DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(NEW,CATLG,DELETE),
000007 //          UNIT=SYSDA,VOL=SER=USRVS1,
000008 //          SPACE=(TRK,(5,2),RLSE),
          //          DCB=(DSORG=PS,RECFM=FB,
***** Bottom of Data *****

```

Allocate New Data Set

Command ==> _____

Data Set Name . . . : IBMUSER.QSAM1

Management class . . .	_____	(Blank for default management class)
Storage class	SCBASE	(Blank for default storage class)
Volume serial	USRVS1	(Blank for system default volume) **
Multiple Volumes	_____	(Enter '/' to select option)
Device type	_____	(Generic unit or device address) **
Data class	_____	(Blank for default data class)
Space units	TRACK	(BLKS, TRKS, CYLS, KB, MB, BYTES or RECORDS)
Average record unit	_____	(M, K, or U)
Primary quantity . .	5	(In above units)
Secondary quantity	2	(In above units)
Directory blocks . .	0	(Zero for sequential data set) *
Record format	FB	
Record length	80	
Block size	16000	
Data set name type	_____	(LIBRARY, PDS, LARGE, BASIC, * EXTREQ, EXTPREF or blank)

Record format "FB" for "fixed length records, blocked" corresponds to the DD DCB subparameter RECFM=.

ATTRIBUTES OF A SEQUENTIAL DATA SET

```

File Edit Edit_Settings Menu Utilities Compilers Test H
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01      Colum
Command ==>                                     Sc
***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1        DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(NEW,CATLG,DELETE),
000007 //          UNIT=SYSDA,VOL=SER=USRVS1,
000008 //          SPACE=(TRK,(5,2),RLSE),
          //          DCB=(DSORG=PS,RECFM=FB LRECL=80,
***** Bottom of Data *****

```

Record length corresponds to the DCB subparameter, LRECL=. That stands for "logical record length".

```

Allocate New Data Set
Command ==>
Data Set Name . . . : IBMUSER.QSAM1
Management class . . . (Blank for default management class)
Storage class . . . SCBASE (Blank for default storage class)
Volume serial . . . USRVS1 (Blank for system default volume) **
Multiple Volumes - (Enter '/' to select option)
Device type . . . (Generic unit or device address) **
Data class . . . (Blank for default data class)
Space units . . . TRACK (BLKS, TRKS, CYLS, KB, MB, BYTES
or RECORDS)
Average record unit - (M, K, or U)
Primary quantity . . 5 (In above units)
Secondary quantity . 2 (In above units)
Directory blocks . . 0 (Zero for sequential data set) *
Record format . . . FB
Record length . . . 80
Block size . . . 16000
Data set name type - (LIBRARY, PDS, LARGE, BASIC, *
EXTREQ, EXTPREF or blank)

```


ATTRIBUTES OF A SEQUENTIAL DATA SET

```

File Edit Edit_Settings Menu Utilities Compilers Test H
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01      Colum
Command ==>                                     Sc
***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1        DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(NEW,CATLG,DELETE),
000007 //          UNIT=SYSDA,VOL=SER=USRVS1,
000008 //          SPACE=(TRK,(5,2),RLSE),
          //          DCB=(DSORG=PS,RECFM=FB LRECL=80,
***** Bottom of Data *****

```

Record length corresponds to the DCB subparameter, LRECL=. That stands for "logical record length".

```

Allocate New Data Set
Command ==>
Data Set Name . . . : IBMUSER.QSAM1
Management class . . . (Blank for default management class)
Storage class . . . SCBASE (Blank for default storage class)
Volume serial . . . USRVS1 (Blank for system default volume) **
Multiple Volumes - (Enter '/' to select option)
Device type . . . (Generic unit or device address) **
Data class . . . (Blank for default data class)
Space units . . . TRACK (BLKS, TRKS, CYLS, KB, MB, BYTES
or RECORDS)
Average record unit - (M, K, or U)
Primary quantity . . 5 (In above units)
Secondary quantity . 2 (In above units)
Directory blocks . . 0 (Zero for sequential data set) *
Record format . . . FB
Record length . . . 80
Block size . . . 16000
Data set name type - (LIBRARY, PDS, LARGE, BASIC, *
EXTREQ, EXTPREF or blank)

```

ATTRIBUTES OF A SEQUENTIAL DATA SET

The image shows two ISPF panels. The left panel is an editor window titled 'EDIT IBMUSER.LAB.JCL(CRQSAM) - 01.01'. It displays JCL code for a sequential dataset. The right panel is the 'Allocate New Data Set' dialog, which mirrors the dataset attributes defined in the JCL.

JCL Code (Left Panel):

```

000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1        DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(NEW,CATLG,DELETE),
000007 //          UNIT=SYSDA,VOL=SER=USRV51,
000008 //          SPACE=(TRK,(5,2),RLSE),
          //          DCB=(DSORG=PS,RECFM=FB,LRECL=80, BLKSIZE=16000)
  
```

Allocate New Data Set Panel (Right Panel):

Field	Value	Description
Data Set Name	IBMUSER.QSAM1	
Management class		(Blank for default management class)
Storage class	SCBASE	(Blank for default storage class)
Volume serial	USRV51	(Blank for system default volume) **
Multiple Volumes		(Enter '/' to select option)
Device type		(Generic unit or device address) **
Data class		(Blank for default data class)
Space units	TRACK	(BLKS, TRKS, CYLS, KB, MB, BYTES or RECORDS)
Average record unit		(M, K, or U)
Primary quantity	5	(In above units)
Secondary quantity	2	(In above units)
Directory blocks	0	(Zero for sequential data set) *
Record format	FB	
Record length	80	
Block size	16000	
Data set name type		(LIBRARY, PDS, LARGE, BASIC, * EXTREQ, EXTPREF or blank)

Block size on the ISPF panel corresponds to the BLKSIZE= subparameter of the DCB= parameter of the DD statement in JCL.

Since the logical records are of fixed length, the block size can be any even multiple of the logical record length. There's an installation-defined maximum. If we specify a value larger than that, the system will silently substitute the largest value below the maximum that works for the given logical record length.

RUNNING THE JOB

```
File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01      Columns 00001 00072
Command ==>                               Scroll ==> CSR
***** Top of Data *****
000001 //IBMUSER1 JOB , 'ALLOC QSAM',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1        DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(NEW,CATLG,DELETE),
000007 //          UNIT=SYSDA,VOL=SER=USRV1,
000008 //          SPACE=(TRK,(5,2),RLSE),
000009 //          DCB=(DSORG=PS,RECFM=FB,LRECL=80,BLKSIZE=16000)
***** Bottom of Data *****
```

Almost forgot to change the text on the JOB statement from 'DO NOTHING' to something more meaningful.
The little things matter.

RUNNING THE JOB

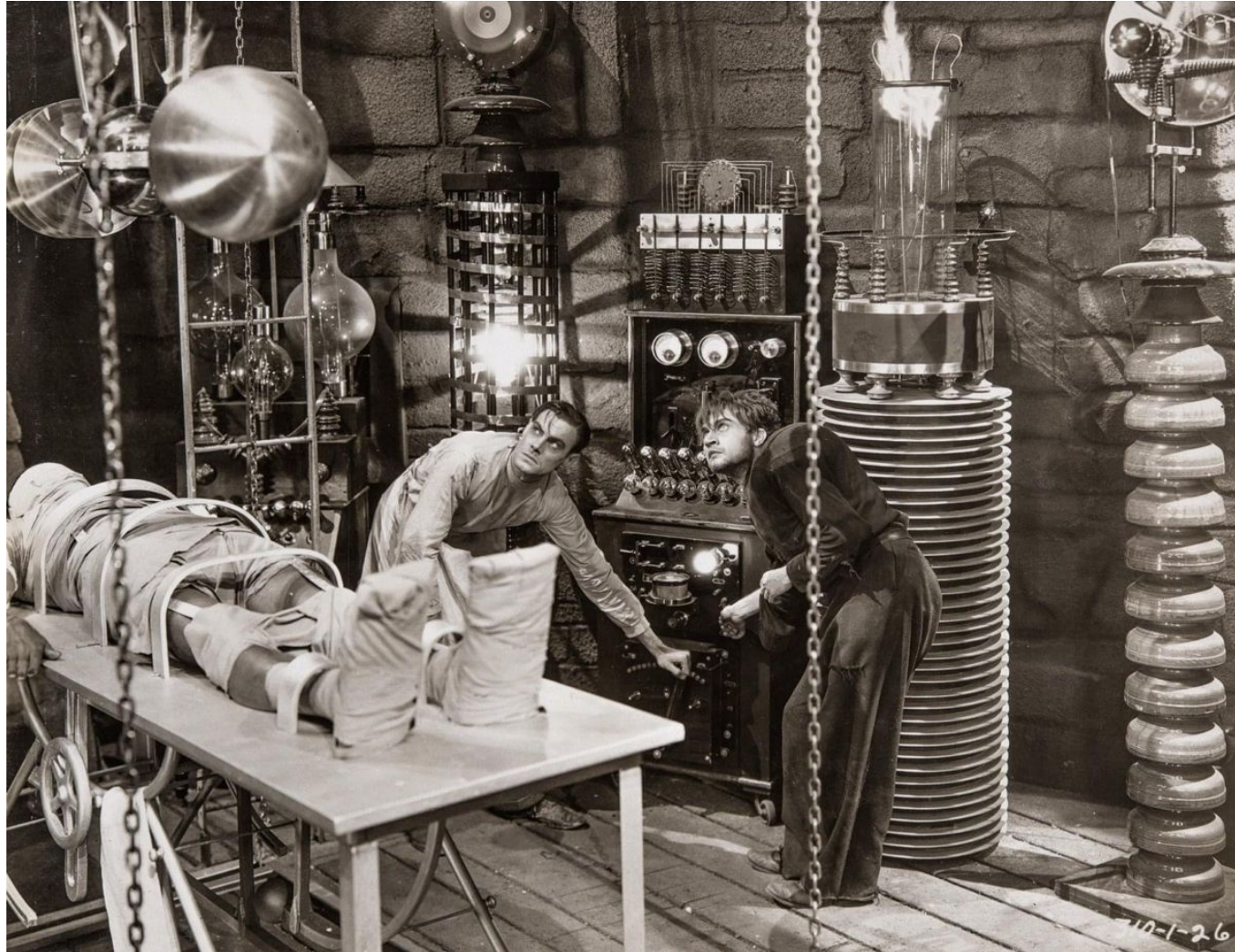
```
Display Filter View Print Options Search Help
-----
SDSF JOB DATA SET DISPLAY - JOB IBMUSER1 (JOB01690) LINE 1-3 (3)
COMMAND INPUT ==>
NP DDNAME StepName ProcStep DSID Owner C Dest
JESMSG LG JES2 2 IBMUSER X LOCAL
JESJCL JES2 3 IBMUSER X LOCAL
JESYSMSG JES2 4 IBMUSER X LOCAL
```

When we run the job we get condition code zero and the three standard JES-generated output data sets.

The interesting one to look at is JESYSMSG. It shows the step ran with condition code zero, data set IBMUSER.QSAM1 was allocated and resides on volume serial number USRVS1, and it was retained, meaning it still exists after the job completed.

```
Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSER1 JOB01690 DSID 4 LINE 0 COLUMNS 02- 81
COMMAND INPUT ==>
***** TOP OF DATA *****
ICH70001I IBMUSER LAST ACCESS AT 13:38:30 ON TUESDAY, JUNE 9, 2026
IEFA111I IBMUSER1 IS USING THE FOLLOWING JOB RELATED SETTINGS:
        SWA=BELOW,TIOT SIZE=64K,DSENQSHR=DISALLOW,GDGBIAS=JOB
IEF236I ALLOC. FOR IBMUSER1 ALLOC
IGD101I SMS ALLOCATED TO DDNAME (DD1 )
        DSN (IBMUSER.QSAM1 )
        STORCLAS (SCBASE) MGMTCLAS ( ) DATACLAS ( )
        VOL SER NOS= USRVS1
IEF142I IBMUSER1 ALLOC - STEP WAS EXECUTED - COND CODE 0000
IGD104I IBMUSER.QSAM1 RETAINED, DDNAME=DD1
IEF373I STEP/ALLOC /START 2026160.1434
IEF032I STEP/ALLOC /STOP 2026160.1434
        CPU: 0 HR 00 MIN 00.00 SEC SRB: 0 HR 00 MIN 00.00 SEC
        VIRT: 8K SYS: 296K EXT: 0K SYS: 9872K
        ATB- REAL: 1252K SLOTS: 0K
        VIRT- ALLOC: 14M SHRD: 0M
IEF375I JOB/IBMUSER1/START 2026160.1434
IEF033I JOB/IBMUSER1/STOP 2026160.1434
        CPU: 0 HR 00 MIN 00.00 SEC SRB: 0 HR 00 MIN 00.00 SEC
***** BOTTOM OF DATA *****
```


LAB Z/OS 9: CREATE A SEQUENTIAL DATA SET



RUNNING THE JOB AGAIN

```
File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01      Columns 00001 00072
Command ==>                               Scroll ==> CSR
***** Top of Data *****
000001 //IBMUSER1 JOB , 'ALLOC QSAM',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //ALLOC      EXEC PGM=IEFBR14
000005 //DD1        DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(NEW,CATLG,DELETE),
000007 //          UNIT=SYSDA,VOL=SER=USRVS1,
000008 //          SPACE=(TRK,(5,2),RLSE),
000009 //          DCB=(DSORG=PS,RECFM=FB,LRECL=80,BLKSIZE=16000)
***** Bottom of Data *****
```

What happens if we run the same job a second time without changing anything?

RUNNING THE JOB AGAIN

The job will only work once. Every time we try to allocate the data set, JES detects the fact it already exists.

Sometimes we need to get rid of the old to make way for the new.

You might be running a job that provisions a development environment for new employees. They might need a number of Libraries to be created fresh, and possibly loaded with default members.

That type of job needs to be idempotent. That is, no matter how many times you run it, the end result is the same as if you had run it exactly once.

To accomplish that, we need a preliminary step before the ALLOC step, to delete any data sets that may be left over from past work and that we want to create fresh.

```

Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSER1 JOB01691 DSID      4 LINE 0      COLUMNS 02- 81
COMMAND INPUT ==> |                               SCROLL ==> CSR
***** TOP OF DATA *****
ICH70001I IBMUSER  LAST ACCESS AT 14:34:17 ON TUESDAY, JUNE 9, 2026
IEFA111I IBMUSER1 IS USING THE FOLLOWING JOB RELATED SETTINGS:
          SWA=BELOW,TIOT SIZE=64K, DSENQSHR=DISALLOW, GDGBIAS=JOB
IEF344I IBMUSER1 ALLOC DD1 - ALLOCATION FAILED DUE TO DATA FACILITY SYSTEM ERROR
IGD17001I DUPLICATE DATA SET NAME ON VOLUME USRV51
FOR DATA SET IBMUSER.QSAM1
IGD17273I ALLOCATION HAS FAILED FOR ALL VOLUMES SELECTED FOR DATA SET
IBMUSER.QSAM1
IGD17277I THERE ARE (215) CANDIDATE VOLUMES OF WHICH (1) ARE ENABLED OR QUIESCED
IGD17290I THERE WERE 2 CANDIDATE STORAGE GROUPS OF WHICH THE FIRST 2
WERE ELIGIBLE FOR VOLUME SELECTION.
THE CANDIDATE STORAGE GROUPS WERE: SGBASE SGEXTEAV
IGD17279I 214 VOLUMES WERE REJECTED BECAUSE THEY WERE NOT ONLINE
IGD17279I 214 VOLUMES WERE REJECTED BECAUSE THE UCB WAS NOT AVAILABLE
IGD17279I 1 VOLUMES WERE REJECTED BECAUSE OF DUPLICATE DATA SET NAME (041C0416)
IEF272I IBMUSER1 ALLOC - STEP WAS NOT EXECUTED.
IEF373I STEP/ALLOC  /START 2026160.1443
IEF032I STEP/ALLOC  /STOP  2026160.1443
          CPU:      0 HR  00 MIN  00.00 SEC      SRB:      0 HR  00 MIN  00.00 SEC
          VIRT:      0K  SYS:      0K  EXT:      0K  SYS:      0K
          ATB- REAL:      1248K      SLOTS:      0K
          VIRT- ALLOC:      14M SHRD:      0M
IEF375I JOB/IBMUSER1/START 2026160.1443
IEF033I JOB/IBMUSER1/STOP  2026160.1443
          CPU:      0 HR  00 MIN  00.00 SEC      SRB:      0 HR  00 MIN  00.00 SEC
***** BOTTOM OF DATA *****
  
```

DELETE THE OLD BEFORE CREATING THE NEW

```
File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01          Columns 00001 00072
Command ==>                                     Scroll ==> CSR
***** ***** Top of Data *****
000001 //IBMUSER1 JOB , 'ALLOC QSAM',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //CLEANUP EXEC PGM=IEFBR14
000005 //DD1      DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(OLD,DELETE,DELETE)
000007 //ALLOC   EXEC PGM=IEFBR14
000008 //DD1      DD DSN=&SYSUID..QSAM1,
000009 //          DISP=(NEW,CATLG,DELETE),
000010 //          UNIT=SYSDA,VOL=SER=USRV51,
000011 //          SPACE=(TRK,(5,2),RLSE),
000012 //          DCB=(DSORG=PS,RECFM=FB,LRECL=80,BLKSIZE=16000)
***** ***** Bottom of Data *****
```

Here we've added a job step named CLEANUP that deletes the existing version of data set IBMUSER.QSAM1. We don't need to write a program because JES will act on the DISP= parameter values on the DD statement.

RUNNING THE REVISED JOB

Looks good! The job deleted the old IBMUSER.QSAM1 and created a new one.

Just what we wanted.

DISP=(OLD,DELETE,DELETE) tells JES we want an exclusive lock on the data set and that we want to delete it whether the job step succeeds or fails.

But...it only worked because we *knew* IBMUSER.QSAM1 already existed.

What if we aren't sure whether it exists? Can we code the JCL in a way that deletes the data set if it exists, and otherwise proceeds normally?

```

Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSER1 JOB01692 DSID      4 LINE 0      COLUMNS 02- 81
COMMAND INPUT ==> |
***** TOP OF DATA *****
ICH70001I IBMUSER  LAST ACCESS AT 14:43:34 ON TUESDAY, JUNE 9, 2026
IEFA111I IBMUSER1 IS USING THE FOLLOWING JOB RELATED SETTINGS:
        SWA=BELOW,TIOT SIZE=64K,DSENQSHR=DISALLOW,GDGBIAS=JOB
IEF236I ALLOC. FOR IBMUSER1 CLEANUP
IGD103I SMS ALLOCATED TO DDNAME DD1
IEF142I IBMUSER1 CLEANUP - STEP WAS EXECUTED - COND CODE 0000
IGD105I IBMUSER.QSAM1                                DELETED, DDNAME=DD1
IEF373I STEP/CLEANUP /START 2026160.1451
IEF032I STEP/CLEANUP /STOP 2026160.1451
        CPU:      0 HR  00 MIN  00.00 SEC      SRB:      0 HR  00 MIN  00.00 SEC
        VIRT:      8K SYS:   296K EXT:      0K SYS:   9872K
        ATB- REAL:                1252K SLOTS:                0K
        VIRT- ALLOC:      14M SHRD:      0M
IEF236I ALLOC. FOR IBMUSER1 ALLOC
IGD101I SMS ALLOCATED TO DDNAME (DD1      )
        DSN (IBMUSER.QSAM1
        STORCLAS (SCBASE) MGMTCLAS (      ) DATACLAS (      )
        VOL SER NOS  USRVS1
IEF142I IBMUSER1 ALLOC - STEP WAS EXECUTED - COND CODE 0000
IGD104I IBMUSER.QSAM1                                RETAINED, DDNAME=DD1
IEF373I STEP/ALLOC /START 2026160.1451
IEF032I STEP/ALLOC /STOP 2026160.1451
        CPU:      0 HR  00 MIN  00.00 SEC      SRB:      0 HR  00 MIN  00.00 SEC
        VIRT:      8K SYS:   296K EXT:      0K SYS:   9872K
        ATB- REAL:                1252K SLOTS:                0K
        VIRT- ALLOC:      14M SHRD:      0M
IEF375I JOB/IBMUSER1/START 2026160.1451
IEF033I JOB/IBMUSER1/STOP 2026160.1451
        CPU:      0 HR  00 MIN  00.00 SEC      SRB:      0 HR  00 MIN  00.00 SEC
***** BOTTOM OF DATA *****

```

DELETE THE OLD BEFORE CREATING THE NEW

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01      Columns 00001 00072
Command ==> |                               Scroll ==> CSR
***** Top of Data *****
000001 //IBMUSER1 JOB , 'ALLOC QSAM',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //CLEANUP EXEC PGM=IEFBR14
000005 //DD1      DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(MOD,DELETE,DELETE),
000007 //          UNIT=SYSDA,VOL=SER=USRVS1,
000008 //          SPACE=(TRK,(1,0)),
000009 //          DCB=(DSORG=PS,RECFM=FB,LRECL=80,BLKSIZE=16000)
000010 //ALLOC    EXEC PGM=IEFBR14
000011 //DD1      DD DSN=&SYSUID..QSAM1,
000012 //          DISP=(NEW,CATLG,DELETE),
000013 //          UNIT=SYSDA,VOL=SER=USRVS1,
000014 //          SPACE=(TRK,(5,2),RLSE),
000015 //          DCB=(DSORG=PS,RECFM=FB,LRECL=80,BLKSIZE=16000)
***** Bottom of Data *****

```

A status of MOD on the DISP= parameter tells JES to append to the end of the dataset if it exists, and to create it if it doesn't exist.

RUNNING THE IMPROVED JOB

The DISP=(MOD,DELETE,DELETE) parameter seems to have worked, at least when the data set already existed.

Let's "manually" delete the data set through ISPF Option 3.2 and then run this job again.

```

Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSER1 JOB01693 DSID 4 LINE 0 COLUMNS 02- 81
COMMAND INPUT ==> | SCROLL ==> CSR
***** TOP OF DATA *****
ICH7000I IBMUSER LAST ACCESS AT 14:51:56 ON TUESDAY, JUNE 9, 2026
IEFA111I IBMUSER1 IS USING THE FOLLOWING JOB RELATED SETTINGS:
        SWA=BELOW, TIOT SIZE=64K, DSENQSHR=DISALLOW, GDGBIAS=JOB
IEF236I ALLOC. FOR IBMUSER1 CLEANUP
IGD103I SMS ALLOCATED TO DDNAME DD1
IEF142I IBMUSER1 CLEANUP - STEP WAS EXECUTED - COND CODE 0000
IGD105I IBMUSER.QSAM1 DELETED, DDNAME=DD1
IEF373I STEP/CLEANUP /START 2026160.1500
IEF032I STEP/CLEANUP /STOP 2026160.1500
        CPU: 0 HR 00 MIN 00.00 SEC SRB: 0 HR 00 MIN 00.00 SEC
        VIRT: 8K SYS: 296K EXT: 0K SYS: 9872K
        ATB- REAL: 1252K SLOTS: 0K
        VIRT- ALLOC: 14M SHRD: 0M
IEF236I ALLOC. FOR IBMUSER1 ALLOC
IGD101I SMS ALLOCATED TO DDNAME (DD1 )
        DSN (IBMUSER.QSAM1 )
        STORCLAS (SCBASE) MGMTCLAS ( ) DATACLAS ( )
        VOL SER NOS= USRVS1
IEF142I IBMUSER1 ALLOC - STEP WAS EXECUTED - COND CODE 0000
IGD104I IBMUSER.QSAM1 RETAINED, DDNAME=DD1
IEF373I STEP/ALLOC /START 2026160.1500
IEF032I STEP/ALLOC /STOP 2026160.1500
        CPU: 0 HR 00 MIN 00.00 SEC SRB: 0 HR 00 MIN 00.00 SEC
        VIRT: 8K SYS: 296K EXT: 0K SYS: 9872K
        ATB- REAL: 1252K SLOTS: 0K
        VIRT- ALLOC: 14M SHRD: 0M
IEF375I JOB/IBMUSER1/START 2026160.1500
IEF033I JOB/IBMUSER1/STOP 2026160.1500
        CPU: 0 HR 00 MIN 00.00 SEC SRB: 0 HR 00 MIN 00.00 SEC
***** BOTTOM OF DATA *****

```

ENSURING THE DATA SET DOESN'T EXIST

```
Menu RefList Utilities Help
Data Set Utility
Option ==>
A Allocate new data set
R Rename entire data set
D Delete entire data set
blank Data set information
C Catalog data set
U Uncatalog data set
S Short data set information
V VSAM Utilities

ISPF Library:
Project . . IBMUSER
Group . . . LAB
Type . . . . JCL

Enter "/" to select option
/ Confirm Data Set Delete

Other Partitioned, Sequential or VSAM Data Set:
Name . . . . . SAM1
Volume Serial . . . (If not cataloged, required for option "C")
Data Set Password . . (If password protected)
```



After using "D" to delete the data set and then "blank" to view data set information, we see that the data set is not catalogued.

RUNNING THE IMPROVED JOB

The job ran with condition code zero, and JES reported that the data set was first deleted and then allocated. That's not strictly true, as the data set did not exist at the start of the job; but that's how JES reports it in the JESYSMSG data set.

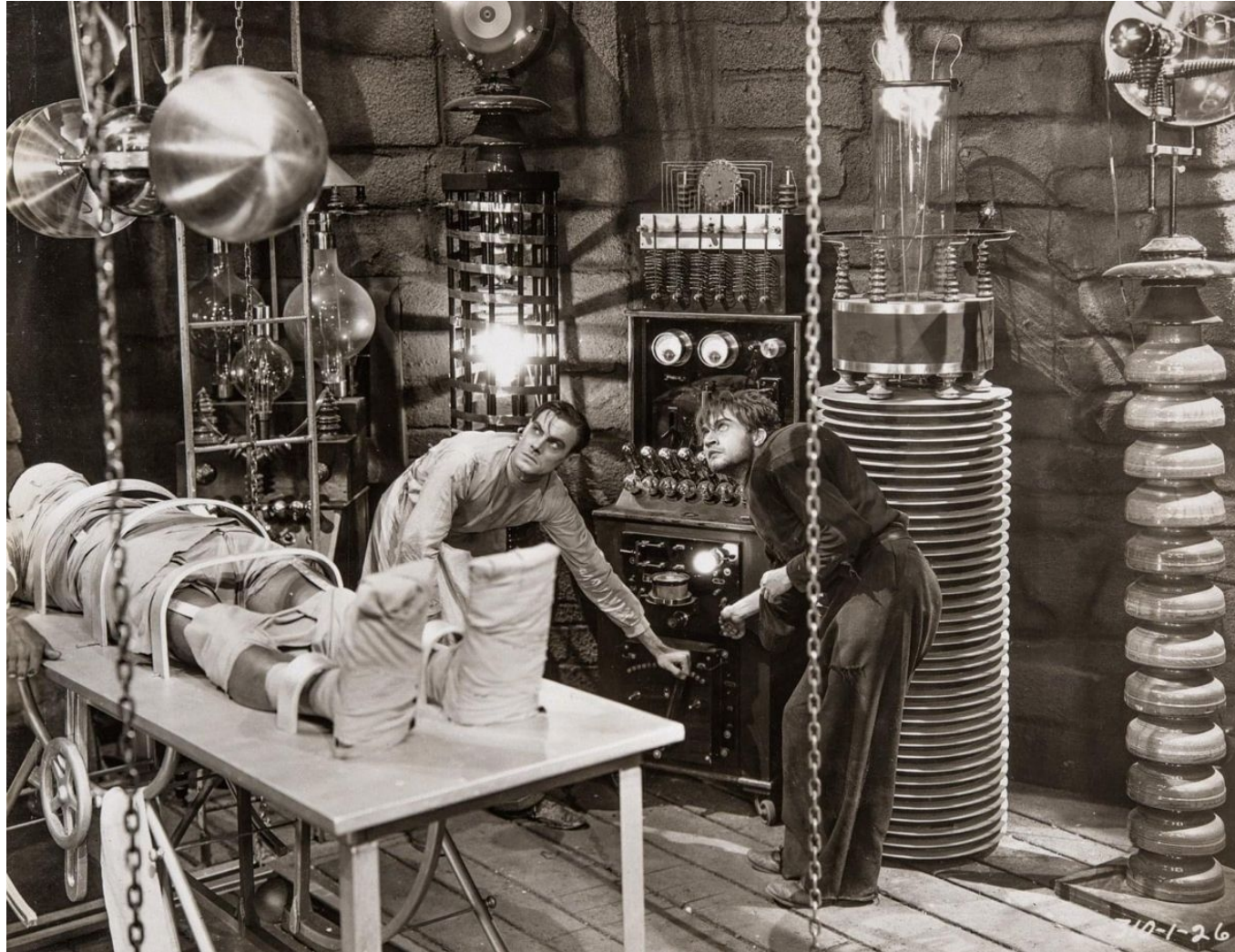
Now we have an idempotent job that cleans up any existing data sets that we want to create fresh, then creates those data sets. Well, one data set.

```

Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSER1 JOB01695 DSID      4 LINE 0      COLUMNS 02- 81
COMMAND INPUT ==> |
***** TOP OF DATA *****
ICH70001I IBMUSER  LAST ACCESS AT 15:05:34 ON TUESDAY, JUNE 9, 2026
IEFA111I IBMUSER1 IS USING THE FOLLOWING JOB RELATED SETTINGS:
          SWA=BELOW,TIOT SIZE=64K,DSENQSHR=DISALLOW,GDGBIAS=JOB
IEF236I ALLOC. FOR IBMUSER1 CLEANUP
IGD101I SMS ALLOCATED TO DDNAME (DD1      )
          DSN (IBMUSER.QSAM1              )
          STORCLAS (SCBASE) MGMTCLAS (      ) DATACLAS (      )
          VOL SER NOS= IISRV51
IEF142I IBMUSER1 CLEANUP - STEP WAS EXECUTED - COND CODE 0000
IGD105I IBMUSER.QSAM1                      DELETED, DDNAME=DD1
IEF373I STEP/CLEANUP /START 2026160.1510
IEF032I STEP/CLEANUP /STOP 2026160.1510
          CPU:      0 HR 00 MIN 00.00 SEC      SRB:      0 HR 00 MIN 00.00 SEC
          VIRT:     8K SYS: 296K EXT:      0K SYS: 9872K
          ATB- REAL:      1252K SLOTS:      0K
          VIRT- ALLOC:    14M SHRD:      0M
IEF236I ALLOC. FOR IBMUSER1 ALLOC
IGD101I SMS ALLOCATED TO DDNAME (DD1      )
          DSN (IBMUSER.QSAM1              )
          STORCLAS (SCBASE) MGMTCLAS (      ) DATACLAS (      )
          VOL SER NOS= IISRV51
IEF142I IBMUSER1 ALLOC - STEP WAS EXECUTED - COND CODE 0000
IGD104I IBMUSER.QSAM1                      RETAINED, DDNAME=DD1
IEF373I STEP/ALLOC /START 2026160.1510
IEF032I STEP/ALLOC /STOP 2026160.1510
          CPU:      0 HR 00 MIN 00.00 SEC      SRB:      0 HR 00 MIN 00.00 SEC
          VIRT:     8K SYS: 296K EXT:      0K SYS: 9872K
          ATB- REAL:      1252K SLOTS:      0K
          VIRT- ALLOC:    14M SHRD:      0M
IEF375I JOB/IBMUSER1/START 2026160.1510
IEF033I JOB/IBMUSER1/STOP 2026160.1510

```

LAB Z/OS 10: MAKE THE JOB IDEMPOTENT



SIMPLIFY THE JCL

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(CRQSAM) - 01.01      Columns 00001 00072
Command ==> |                               Scroll ==> CSR
***** Top of Data *****
000001 //IBMUSER1 JOB , 'ALLOC QSAM',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //CLEANUP EXEC PGM=IEFBR14
000005 //DD1      DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(MOD,DELETE,DELETE),
000007 //          UNIT=SYSDA,VOL=SER=USRV1,
000008 //          SPACE=(TRK,(1,0)),
000009 //          DCB=(DSORG=PS,RECFM=FB,LRECL=80,BLKSIZE=16000)
000010 //ALLOC EXEC PGM=IEFBR14
000011 //DD1      DD DSN=&SYSUID..QSAM1,
000012 //          DISP=(NEW,CATLG,DELETE),
000013 //          UNIT=SYSDA,VOL=SER=USRV1,
000014 //          SPACE=(TRK,(5,2),RLSE),
000015 //          DCB=(DSORG=PS,RECFM=FB,LRECL=80,BLKSIZE=16000)
***** Bottom of Data *****

```

Because DISP=MOD might require JES to create the data set, JES requires us to specify all the DD statement parameters that apply to new data sets. It doesn't notice that DISP=(MOD,DELETE,DELETE) will cause the data set to be deleted regardless of what happens in the job step.

As a result, we have to code the DCB attributes twice. There is a way to simplify this so that we can avoid redundant typing, and also avoid having to change JCL in multiple places if the attributes have to change in future.

MODEL DATA SET CONTROL BLOCK

```
DCB=(DSORG=PS,RECFM=FB,LRECL=80,BLKSIZE=16000)
```

This portion of the DD statements for the QSAM1 data set is identical in all the DD statements in job steps that allocate, or *might* allocate, the data set. We can store this specification and re-use it on multiple DD statements.

It might seem unnecessary just to avoid coding the DCB= parameter twice, but consider the fact many data sets in a given z/OS environment will have similar DCB attributes.

What we want to do is create a catalog entry that contains the DCB information, but not create an actual data set that will contain records. Then we can refer to the catalog entry in DCB= parameters that need to specify that particular set of attributes for new data sets.

MODEL DATA SET CONTROL BLOCK

This JCL is similar to the JCL we used to create sequential data set QSAM1. In this example, the space allocation for the data set is zero. This will result in a catalog entry for data set name IBMUSER.MODELDCB.DCBSEQ80.

The rationale for the data set name is that the mid-level qualifier identifies it as a model DSCB and the low-level qualifier suggests what the DCB attributes are. "SEQ" for "sequential data set" and "80" for 80-byte records.

The length of names is pretty limited on z/OS for historical reasons, but in context the name is reasonable since 80-byte fixed-length records are used for so many things on the platform. It's a throwback to the days of 80-column punched cards.

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(DCBSEQ80) - 01.00      Columns 00001 00072
Command ==> |                               Scroll ==> CSR
***** Top of Data *****
000001 //IBMUSERM JOB , 'MODEL DCB SEQ80',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //CLEANUP EXEC PGM=IEFBR14
000005 //DD1      DD DSN=&SYSUID.MODELDCB.DCBSEQ80,
000006 //          DISP=(MOD,DELETE,DELETE),
000007 //          UNIT=SYSDA,VOL=SER=USRVS1,
000008 //          SPACE=(TRK,(0)),
000009 //          DCB=(DSORG=PS,RECFM=FB,LRECL=80,BLKSIZE=16000)
000010 //ALLOC    EXEC PGM=IEFBR14
000011 //DD1      DD DSN=&SYSUID.MODELDCB.DCBSEQ80,
000012 //          DISP=(NEW,CATLG,DELETE),
000013 //          UNIT=SYSDA,VOL=SER=USRVS1,
000014 //          SPACE=(TRK,(0)),
000015 //          DCB=(DSORG=PS,RECFM=FB,LRECL=80,BLKSIZE=16000)
***** Bottom of Data *****

```


MODEL DATA SET CONTROL BLOCK

After running the job, the data set information looks like this. There are no extents – that is, the data set does not occupy any DASD space.

We can refer to a data set name in the DCB= parameter of a DD statement, and JES will use the DCB attributes of that data set for the one named on the DD statement.

```

Data Set Information
Command ==>

Data Set Name . . . . : IBMUSER.MODELDCB.DCBSEQ80

General Data
Management class . . : **None**
Storage class . . . . : SCBASE
Volume serial . . . . : USRVS1
Device type . . . . . : 3390
Data class . . . . . : **None**
Organization . . . . . : PS
Record format . . . . : FB
Record length . . . . : 80
Block size . . . . . : 16000
1st extent tracks . . : 0
Secondary tracks . . . : 0
Data set name type :
Data set encryption : NO

Current Allocation
Allocated tracks . . : 0
Allocated extents . . : 0

Current Utilization
Used tracks . . . . . : 0
Used extents . . . . . : 0

Dates
Creation date . . . . : 2026/06/09
Referenced date . . . : ***None***
Expiration date . . . . : ***None***

SMS Compressible . . : NO
  
```

MODEL DATA SET CONTROL BLOCK

With the model DSCB catalogued, we can change the CRQSAM job to refer to the model instead of hand-coding all the DCB attributes every time.

In this example, I made a copy of the original JCL under the name CRQSAM2 so I would have the first version available as a reference.

The DCB= parameters on the DD statements now give the name of the catalog entry for the model DSCB instead of specifying the attributes directly.

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(CRQSAM2) - 01.00      Columns 00001 00072
Command ==>                               Scroll ==> CSR
***** Top of Data *****
000001 //IBMUSER2 JOB , 'ALLOC QSAM',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //CLEANUP EXEC PGM=IEFBR14
000005 //DD1      DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(MOD,DELETE,DELETE),
000007 //          UNIT=SYSDA,VOL=SER=USRVS1,
000008 //          SPACE=(TRK,(1,0))
000009 //          DCB=(&SYSUID..MODELDCB.DCBSEQ80)
000010 //ALLOC EXEC PGM=IEFBR14
000011 //DD1      DD DSN=&SYSUID..QSAM1,
000012 //          DISP=(NEW,CATLG,DELETE),
000013 //          UNIT=SYSDA,VOL=SER=USRVS1,
000014 //          SPACE=(TRK,(5,2),RLSE),
000015 //          DCB=(&SYSUID..MODELDCB.DCBSEQ80)
***** Bottom of Data *****

```

MODEL DATA SET CONTROL BLOCK

With the model DSCB catalogued, we can change the CRQSAM job to refer to the model instead of hand-coding all the DCB attributes every time.

In this example, I made a copy of the original JCL under the name CRQSAM2 so I would have the first version available as a reference.

The DCB= parameters on the DD statements now give the name of the catalog entry for the model DSCB instead of specifying the attributes directly.

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(CRQSAM2) - 01.00      Columns 00001 00072
Command ==>                               Scroll ==> CSR
***** ***** Top of Data *****
000001 //IBMUSER2 JOB , 'ALLOC QSAM',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //CLEANUP EXEC PGM=IEFBR14
000005 //DD1      DD DSN=&SYSUID..QSAM1,
000006 //          DISP=(MOD,DELETE,DELETE),
000007 //          UNIT=SYSDA,VOL=SER=USRVS1,
000008 //          SPACE=(TRK,(1,0))
000009 //          DCB=(&SYSUID..MODELDCB.DCBSEQ80)
000010 //ALLOC    EXEC PGM=IEFBR14
000011 //DD1      DD DSN=&SYSUID..QSAM1,
000012 //          DISP=(NEW,CATLG,DELETE),
000013 //          UNIT=SYSDA,VOL=SER=USRVS1,
000014 //          SPACE=(TRK,(5,2),RLSE),
000015 //          DCB=(&SYSUID..MODELDCB.DCBSEQ80)
***** ***** Bottom of Data *****

```

WHAT YOU KNOW AND WHAT YOU CAN DO

- You can write an idempotent job that creates a sequential data set.
- You can create a catalog entry to use as a model data set control block.

WHAT'S NEXT

- Provisioning source libraries for an application developer